

Test Adaptation Framework for Out-of-Tree (OOT) Devices: An IBM Spyre Case Study

*Submitted to the ISSTA 2026 REBASE Industry Track
by*

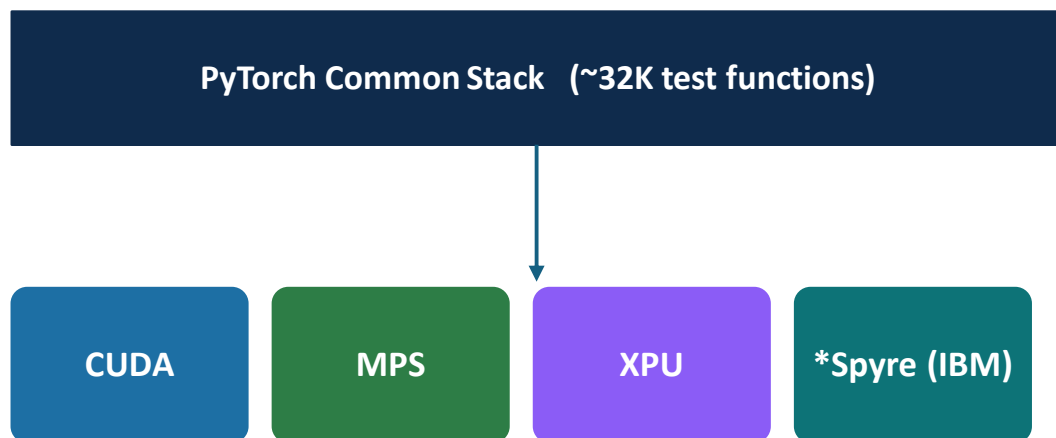
Anubhav Jana, Harissh Ragav Dhamodaran, Padmanabha V Seshadri, Mehant Kammakomati, Ashok Pon Kumar Sree Prakash, Praveen Jayachandran

IBM Research, India

Introduction

PyTorch's plugin-based device architecture and the test reuse problem

PyTorch Ecosystem



The Scale Challenge

Open-source quality can only be realized if the upstream test suite can be reused across all device plugins.

Each new device team faces the same problem: how to run thousands of tests written for pytorch in-tree device like CUDA on an OOT device that doesn't exist in upstream.

Three Facets of the Adaptation Problem

(a) Trivial with no change needed - Run as-is

Some tests run on OOT device as-is. The device is called correctly through PyTorch's dispatch mechanism and all ops + dtypes of those test variants pass successfully.

(b) Decorator + Parameter adaptation

The test logic is correct, but decorators hardcode device specific parameters — e.g. `@onlyOn('cuda')`, unsupported op lists or dtypes. Needs parameter adaptation on the fly

c) In-body function code refactoring

Test body contains hardcoded device strings, CUDA branches, or dtype assumptions that require upstream code changes.

Gap: No framework exists to discover, configure, and apply these adaptations systematically without modifying upstream.

Motivation

Why repurpose upstream PyTorch tests for Spyre?

The Opportunity

- PyTorch ships **~32,000** test functions
- These collect to **~500,000 parametrized** test variants
- Each test encodes correctness expectations for ops, dtypes, and modules
- OOT devices (**PrivateUse1**) all face the same adaptation problem

Example: @onlyOn('cuda') in upstream

```
@onlyOn('cuda')
def test_broadcast_shapes(self, device):
    ...
```

-> Spyre is blocked from running this test
-> Our framework: patches decorator to also allow 'spyre'
-> No upstream change required

Repurposing vs. Rewriting

Rewrite custom tests from scratch

Requires dev cost, quality risk and a probable drift from upstream test infrastructure

Upstream refactoring

Slow process -- waiting for community and might have incomplete coverage

Our approach: Adapt Framework

Instant with zero upstream changes, config-driven YAML-controlled

Value Proposition

Why this matters for OOT accelerator teams

Effort Reduction

Reuses thousands of upstream correctness tests with no need of writing equivalent OOT specific custom tests from scratch.

Faster Iteration

New ops or dtype support in the custom device activates immediately -- **update** the YAML config and the relevant test variants run.

Quality Parity

Learning confidence against the same test infrastructure that defines correctness for PyTorch and primary signal of progress

Evolvability

As both custom device and upstream evolve, adaptation configs **update independently** with no upstream repo commits required.

Related Work

Existing approaches don't address OOT test adaptation

MR-Coupler [FSE 2026]

Generates metamorphic tests via functional coupling analysis. Targets test generation, not reuse of existing upstream parametrized tests.

TestCaseMig

Migrates test cases across evolving codebases using LLMs. Assumes co-evolution of test and code in the same repo not applicable to OOT plugins.

E-Test [ICSE 2026]

Mines production execution scenarios to augment test suites. Focused on coverage expansion, not device-specific adaptation of decorator-heavy tests.

Regression Test Selection (Patent)

Selects which tests to run via code-change analysis. Addresses scheduling but not the semantic adaptation of decorator arguments for new devices.

Gap: No prior work addresses declarative and a non-invasive adaptation of upstream tests for OOT plugin devices.

Challenge 1: Test Discovery

Which upstream tests are relevant now and which become relevant as Spyre evolves?

32K+ upstream test functions

Manually reviewing each is infeasible. Need automated relevance ranking.

OOT code evolves

A test irrelevant today (e.g. op not supported) may become relevant next sprint. Discovery must be re-run continuously.

Device code paths matter

A test is only worth running on custom accelerator only if it actually exercises the OOT device code path and not just CPU fallback.

Test Lifecycle Over Time



Challenge 2a: Test Adaptation (Decorator-Based)

Upstream decorators block OOT devices at collection time

Two kinds of decorator problems:

(1) The decorator itself excludes our device

```
@onlyOn('cuda')
@onlyNativeDeviceTypes
@ops(binary_ufuncs)
@dtypes(torch.float32)
```

Problem: @onlyOn reads device_type at call time. E.g. If 'spyre' is not in the list, test is skipped — never even collected.

Solution: Mutate decorator instance in-place — add 'spyre' before instantiate_test() fires.

`_OOTOnlyOnPatcher` | `_OOTNativeDeviceTypesPatcher`

(2) The decorator's inputs restrict test variants

```
@ops(binary_ufuncs, allowed_dtypes=(float32,))
# 200+ ops, only float32
```

Problem: op_list has 200+ ops but Spyre supports a lot less than upstream's **op_db** (op registry) list.

allowed_dtypes=(float32,) but we need float16 too. Wrong variants are generated.

Solution: Access @ops instance via parametrize_fn and filter op_list / expand allowed_dtypes in-place.

`_OOTOpListPatcher` | `_OOTDtypePatcher`
`_OOTOpDtypeExpander` | `_OOTModuleListPatcher`

Challenge 2b: Test Adaptation (In-Body Code)

Hardcoded values inside test bodies require structural refactoring

Even after decorators are fixed, test bodies itself may contain device-specific assumptions

Hardcoded device strings

Before

```
device = "Cuda:0"  
tensor = torch.zeros(4, device=device)
```



After

```
tensor = torch.zeros(4, device=device)
```

Replace hardcoded 'cuda' with self.device

CUDA-specific branches

Before

```
if device.type == "Cuda":  
    run_specialized_path()
```



After

```
if device.type in ["Cuda", "spyre"]:  
    run_specialized_path()
```

Generalize branches to be accelerator-aware

Dtype assumptions in tensor args

Before

```
# test hardcodes float32 shapes/strides  
t = torch.ones(8, dtype=torch.float32)
```



After

```
YAML config injects custom SampleInputs  
No upstream code change needed
```

*_OOTCpuMovePatcher +
custom SampleInputs from
YAML config*

Note: In-body refactoring must be contributed upstream. Our framework focuses on deferring this via declarative workarounds where possible.

Design Principle 1: Non-Invasive Decorator Patching

*Mutate decorator instances at instantiation time with **ZERO** upstream modification*

PyTorch decorators read configuration at call time, not at decoration time.

Mutating the decorator instance after decoration but before the test runs is sufficient and leaves upstream code unchanged.

Deep copy protection

PyTorch deepcopies the test method before `instantiate_test()`. Each call has fresh decorator instances -- one-time global patches would be lost.

Closure walking

Decorator stacking hides `@onlyOn` under `@suppress_warnings`, `@ops`, etc. We walk the `__wrapped__` chain to find the `onlyOn` instance in any closure cell.

In-place mutation

`val.device_type.append('spyre')` -- list mutation. For `@ops`: `parametrize_fn.__self__.op_list[:] = filtered`. Identity preserved, effect immediate.

parametrize_fn wrapping

For marker injection, we wrap `parametrize_fn` itself so pytest markers are attached to each generated `test_wrapper` before it's installed as a class method.

Design Principle 2: Declarative YAML Configuration

Express all adaptation mechanism as config yaml independent of the upstream test code

test_suite_config.yaml

```
global:
  supported_dtypes:
    - name: float16
    - name: float32
      supported_ops:
    - name: add
      dtypes:
    - name: float16
      precision:
        atol: 1e-3
        rtol: 1e-3

files:
  - path: /path/to/test_binary_ufuncs.py
    tests:
      - names: [test_scalar_support]
        mode: mandatory_success
        edits:
          ops:
            include: [add, mul]
          dtypes:
            include: [float16]
```

global.*

Declares device-wide supported ops, dtypes, modules -- the capability profile of the OOT device.

files[].tests[].mode

mandatory_success | xfail | xfail_strict | skip — controls CI behavior per test.

edits.ops.include/exclude

Filters @ops.op_list to only the ops Spyre supports; prevents collection of thousands of unsupported variants.

edits.dtypes.include

Expands @ops.allowed_dtypes and op.dtypes on each OpInfo -- injects device-specific dtype variants.

edits.precision

Attaches precision_overrides / tolerance_overrides per dtype; allows numerical tolerance tuning without upstream change.

Design Principle 3: Runtime Test Patching Internals

When patching occurs and which upstream test-generation semantics are modified without changing source?

Patching occurs inside our ***OOTestBase.instantiate_test()*** after PyTorch deep-copies the test method, but before `super().instantiate_test()` calls `_parametrize_test` to generate concrete variants.

Sequencing within `instantiate_test()`:

Step 1: Patchers run

Mutate decorator instances in-place on the deep-copied test method.

Step 2: `super().instantiate_test()` runs

Calls `_parametrize_test` to generate all concrete variant methods.

Step 3: Post-generation loop

Applies per-variant mode: `mandatory_success`, `xfail/skip/tag` logic over ***new_methods***.

Upstream semantics modified at collection time (no source changes):

@ops.op_list: filtered in-place to only ops in YAML; unsupported variants are never generated.

@ops.allowed_dtypes: expanded to include extra dtypes from YAML (outer filter) beyond what the upstream decorator declares.

OpInfo.dtypes / OpInfo.dtypesIfPrivateUse1: expanded per-op via `__dict__` direct write to bypass validation (inner filter).

@modules.module_info_list: filtered/expanded to modules declared in YAML.

@modules.allowed_dtypes: same expansion logic as `@ops.allowed_dtypes`, applied to the modules decorator.

@onlyOn: "spyre" and "privateuse1" appended to its `device_type` list so device-restricted tests are no longer skipped.

Why a Static Downstream Capability Matrix + pytest Markers Is Not Enough?

We address a core technical gap: pytest markers filter existing test items; they cannot control which items are generated

This framework intercepts the generation process itself to expand, filter, and annotate variants at (op, dtype, module) granularity without touching a single upstream pytorch test file.

Gap 1: Variants that were never generated

`@ops(binary_ufuncs, allowed_dtypes=(float32,))` produces zero float16 variants. A pytest marker on a non-existent item does nothing. The framework patches `@ops.allowed_dtypes` before `_parametrize_test` runs, injecting float16 so the variant is generated at all.

Gap 3: Variant-level granularity within a single test

A function-level marker xfails ALL variants of a test. The framework operates at (op, dtype, module) variant level. For example, `test_transposes_errors` can be mandatory_success for float16 while excluding int64 --- **impossible to express with a function-level marker**.

Gap 2: Ops excluded by pre-filtered lists

Upstream tests use pre-filtered sublists (e.g. `binary_ufuncs_with_references`). If an op has no ref, its variant is never generated -- **nothing to mark**. The framework patches `@ops.op_list` in-place before generation to inject the op.

Gap 4: Decorator guards that fire before any marker logic

`@onlyOn("cuda")` raises `unittest.SkipTest` before the test body runs **before pytest xfail/skip ever sees the result**. `pytest.mark.xfail` cannot reach these tests. The framework patches the decorator's `device_type` list in-place so the guard passes for `spyre/privateuse1`.

Design Principle 4: Structured Upstream Refactoring

When adaptation isn't enough : contribute generalizations back upstream from our discovery + adaptation pipeline

Some tests cant be adapted purely through decorator patching. These require upstream refactoring which we automate via our test discovery + adaptation framework pipeline (Some examples are below)

Replace @onlyCUDA

`@onlyCUDA → @onlyAccelerator`

Upstream decorator generalization

Generalize @dtypesIfCUDA

`@dtypesIfCUDA(f16) ->
@dtypesIfAccelerator(f16)`

Dtype validation becomes device-aware

Replace hardcoded device strings

`"cuda:0" -> inferred from test fixture`

Enables any registered backend

Generalize device_type branches

`if device == "cuda" -> if device in
accelerators`

Runtime-aware backend dispatching

Refactoring PRs are tracked per-test in the adapted tests catalogue and linked to upstream GitHub commits.

Upstream Test Discovery Pipeline

Pytorch Upstream

Test Rep
32k test
function
s



Query on
tests of
interest

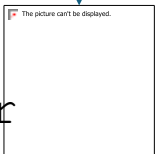
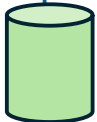
Already discovered tests which could be potentially activated for new features

Test
Discovery
(Claude
skill)

Test Config
Generator
(Claude
skill)

Test
runner

Log
Analyzer
(Claude
skill)



Black list
(tests not relevant to spyre)

White list
(tests marked by module level relevance in PyTorch)

Spyre device documentation
(ops, modules, dtypes, shapes supported by spyre device)

Spyre
cluster

Failed tests

Passed tests

Does not hit
torch-spyre

Ignore

hits
torch-
spyre

**Marked
mandatory
pass**

Does not hit
torch-spyre

Ignore

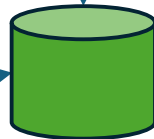
hits torch-
spyre

Unsupported
features

**Marked
skip**

Needs test
refactoring

**Marked skip (and mark
"needs refactoring")**



**Adapted
tests
catalogue**

Natural
language
query
(e.g.
describing
functionalit
y,
downstream
test code)

Github Actions
Trigger
(e.g. new issue,
PR)

Git Issue

Git PR requiring
tests to cover
new code
(E.g new op/dtype

Git PR adding new
tests

Test Discovery: Results (pytorch/test)

Number of test modules (i.e files) shortlisted for spyre	368
Number of test modules (i.e files) processed by the pipeline	210
Number of tests collected	9978
Number of passing tests (those which do not hit spyre are skipped)	1067
Number of passing tests that hit the spyre device	343
Number of xfail candidates (hit the spyre, but throw exception. Either need refactoring to work or require features that are not currently supported by spyre stack)	8295
Number of skipped candidates	616

OOT Adaptation Coverage - Individual Test Listing

[upstream pytorch test dsdiscovery] 37 YAML configs · per-mechanism test-name breakdown

4

`edits.ops.exclude`
(test names, 2
files)

19

`edits.dtypes.exclude`
(test names, 7
files)

110

`global.supported_ops`
(@ops tests, 7
files)

836

`global.supported_dtypes`
(@dtypes/@ops
tests, 19 files)

***per-test edits** (YAML `edits.*`, curated one `TestEntry` at a time)*
***capability filters** (YAML `global.*`, applied file-wide)*

| ***global***

- **253 of 295** decorator-gated are additionally curated with an explicit YAML mode; 42 are unlocked purely by the unconditional patch (`@onlyOn` / `@onlyNativeDeviceTypes`) with zero YAML entry
- See next slide for the full `edits.ops.exclude` / `edits.dtypes.exclude` test-name listing, and next-to-next slide for the `global.supported_ops` / `global.supported_dtypes` per-file breakdown.

***Note:** These numbers are generated at the time of writing the proposal slides and subjected to change we scale up the framework to discover and adapt more test for spyre*

Individual Test Listing: Per-Test Edits

edits.ops.exclude and edits.dtypes.exclude; one row per curated test name

edits.ops.exclude (4 test names, 2 files)

YAML file	Test	Excluded ops
test_binary_ufuncs	TestBinaryUfuncs::test_not_broadcastable	8: add, bitwise_and, clamp_max, clamp_min, maximum ...
test_binary_ufuncs	TestBinaryUfuncs::test_type_promotion	5: add, maximum, minimum, mul, sub
test_reductions	TestReductions::test_dim_multi_duplicate	52: _scaled_dot_product_f used_attention_overrideable, _scaled_mm, abs, add, addmm ...
test_reductions	TestReductions::test_empty_tensor_nonempty_slice	52: _scaled_dot_product_f used_attention_overrideable, _scaled_mm, abs, add, addmm ...

Note: These numbers are generated at the time of writing the proposal slides and subjected to change we scale up the framework to discover and adapt more test for spyre

edits.dtypes.exclude (19 test

YAML file	Test	Excluded dtypes
nn-test_pooling	TestPoolingNNDeviceType::test_pool_invalid_size	bfloat16, bool, float16, int64
test_binary_ufuncs	TestBinaryUfuncs::test_not_broadcastable	bool, int64
test_binary_ufuncs	TestBinaryUfuncs::test_sub	bfloat16, float16, float32, int64
test_shape_ops	TestShapeOps::test_sparse_dense_dim	int64
test_tensor_creation_ops	TestTensorCreation::test_dsplitle	int64
test_tensor_creation_ops	TestTensorCreation::test_float_to_int_conversion_nonfinite	int64
test_tensor_creation_ops	TestTensorCreation::test_hsplit	int64
test_tensor_creation_ops	TestTensorCreation::test_vsplit	int64
test_tensor_creation_ops	TestTensorCreation::test_cat_trailing_dim	bfloat16, bool, float16, int64
test_tensor_creation_ops	TestAsArray::test_retain_autograd_history	bfloat16, bool, float16, int64
test_torch	TestTorchDeviceType::test_addcddiv	float32
test_unary_ufuncs	TestUnaryUfuncs::test_abs_zero	bfloat16, bool, float16, int64
test_view_ops	TestOldViewOps::test_broadcast_to	bool, int64
test_view_ops	TestOldViewOps::test_tensor_split_indices	bool, int64
test_view_ops	TestOldViewOps::test_tensor_split_sections	bool, int64
test_view_ops	TestOldViewOps::test_transposes_errors	bool, int64
test_view_ops	TestViewOps::test_imag_noncomplex	float16, float32
test_view_ops	TestViewOps::test_view_dtype_upsize_errors	bool, int64
test_view_ops	TestViewOps::test_view_tensor_split	bool, int64

Individual Test Listing: Global Capability Filters

Tests decorated with @ops / @dtypes in upstream source, filtered by global.supported_ops / global.supported_dtypes

global.supported_ops (110 @ops tests, 7 files)

YAML file	@ops tests
test_ops	37
test_reductions	35
test_binary_ufuncs	16
test_unary_ufuncs	13
test_spectral_ops	6
test_masked	2
inductor-test_op_dtype_prop	1

global.supported_dtypes

YAML file	tests
test_linalg	209
test_reductions	89
test_binary_ufuncs	88
test_tensor_creation_ops	79
test_torch	78
test_unary_ufuncs	55
test_ops	37
nn-test_convolution	34
test_spectral_ops	33
nn-test_pooling	25

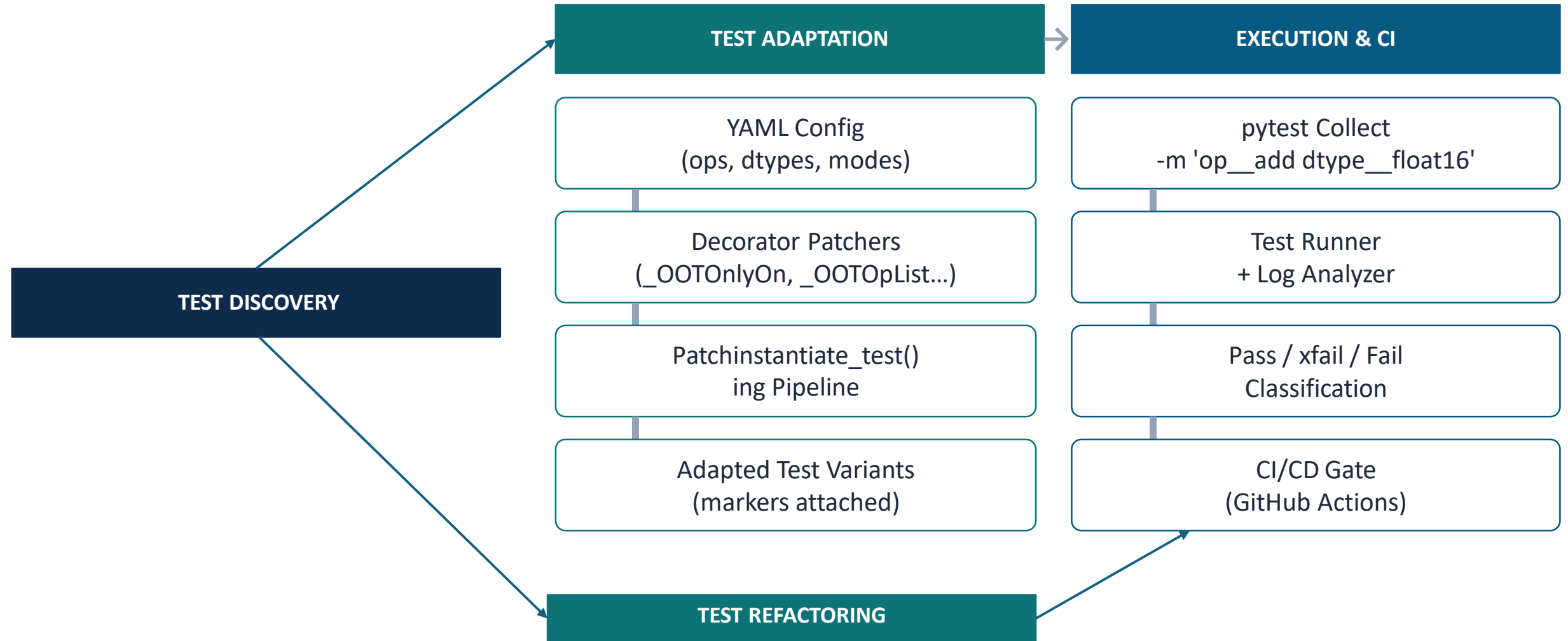
(836 @dtypes/@ops tests, 19

YAML file	tests
test_sort_and_select	22
test_view_ops	22
nn-test_embedding	21
test_shape_ops	17
test_indexing	11
test_prims	11
inductor-test_inductor_scheduler	2
test_masked	2
inductor-test_op_dtype_prop	1

Note: These numbers are generated at the time of writing the proposal slides and subjected to change we scale up the framework to discover and adapt more test for spyre

Test Adaptation Framework - System Overview

End-to-end pipeline from upstream tests to CI/CD-validated results



Framework Block: Test Discovery

Combining static analysis, repo memory, and execution signals

Static Analysis

Parse AST of upstream test files to identify decorator signatures, op names, dtype arguments. Build a searchable index of test–op–dtype relationships.

Repo Memory Agent

LLM agent with memory of upstream + downstream repo. Given a NL query or new PR, retrieves relevant test candidates. Outputs ranked list of tests to activate.

Execution-Based Signal

Run candidates in dry-run mode. Classify by: hits plugin / doesn't hit plugin / fails. Tests that hit plugin code are candidates for adaptation config.

Output: Adapted Tests Catalogue

mandatory_success

Test must pass : failure blocks CI

xfail

Expected failure : CI passes either way

xfail_strict

Must fail : passes CI only if it fails

skip

Never collected : variant deleted entirely

Framework Block: Test Refactoring

Combining syntax validation, hardware-aware execution, and self-correcting retries, the agent refactors Upstream tests with zero manual intervention.

Static Analysis

Parse AST of upstream test files to identify CUDA decorators, hard-coded CUDA calls, and device-bound op signatures. Build a map of what needs replacing before any code is touched.

Refactor Agent

LLM agent with the livesweagent's core loop. Given a target test file, rewrites decorators and device strings, applies CPU offload where needed, and adapts code through pre-check and verification gates.

Syntax & Import Validation

Lint the refactored file for syntax errors, broken imports, and indentation issues before anything runs. Fast, lightweight check that catches malformed edits early and routes failures straight back to Refactor.

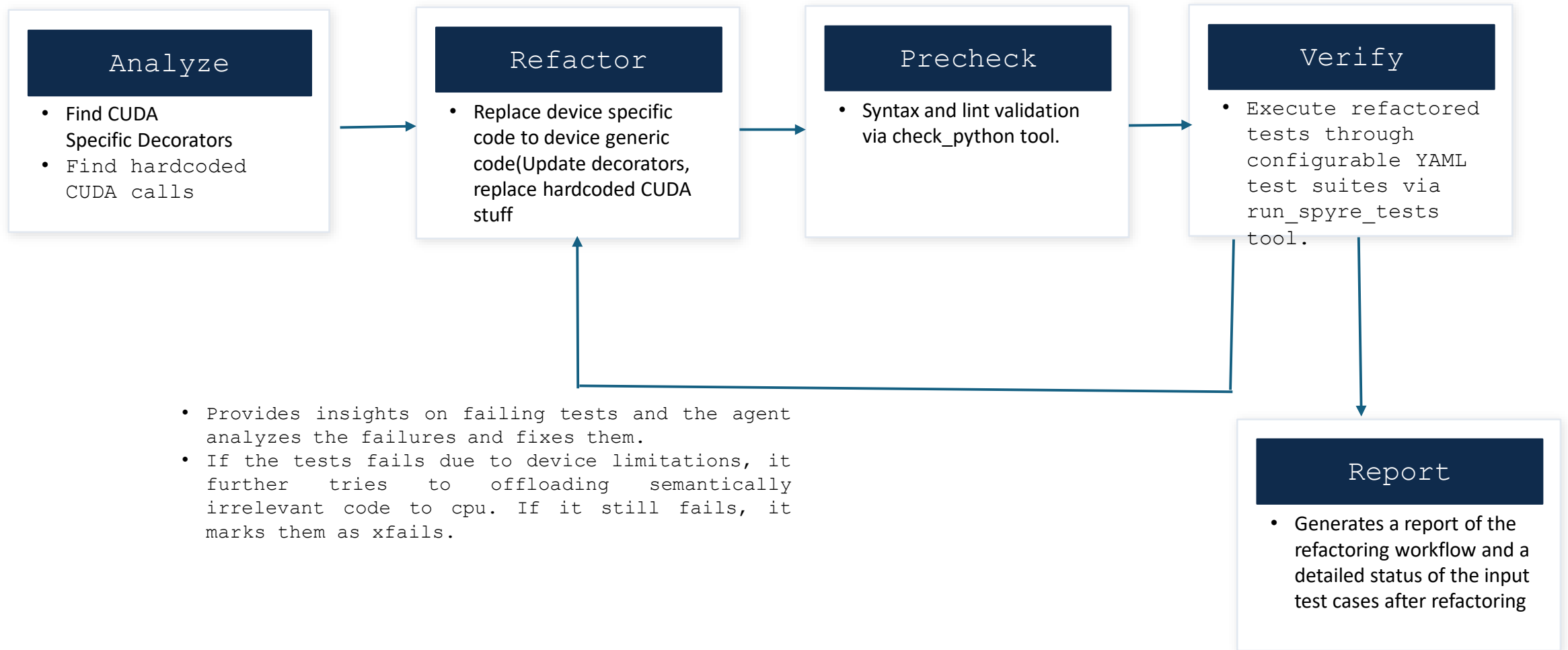
Target Execution

Generate the test suite config and run it against the actual accelerator hardware.

Bash & Terminal Access

Direct shell access for everything outside the two purpose-built tools: running arbitrary commands, inspecting or editing files, and using git/grep/find to navigate the repo while refactoring.

Test Refactoring: Workflow



Test Refactoring: Results

```
@onlyCUDA
def test_cat_stack_cross_devices(self, device):
    cuda = torch.randn((3, 3), device=device)
    cpu = torch.randn((3, 3), device='cpu')

    # Stack
    with self.assertRaisesRegex(RuntimeError,
                                "Expected all tensors to be on the same devi
                                torch.stack((cuda, cpu))
    with self.assertRaisesRegex(RuntimeError,
                                "Expected all tensors to be on the same devi
                                torch.stack((cpu, cuda))

# TODO: replace with other cat tests
```

```
@onlyNativeDeviceTypes
def test_cat_stack_cross_devices(self, device):
    cuda = torch.randn((3, 3), device=device)
    cpu = torch.randn((3, 3), device='cpu')

    # Stack
    with self.assertRaisesRegex(RuntimeError,
                                "Expected all tensors to be on the same device"):
        torch.stack((cuda, cpu))
    with self.assertRaisesRegex(RuntimeError,
                                "Expected all tensors to be on the same device"):
        torch.stack((cpu, cuda))

# TODO: replace with other cat tests
```

Automated Decorator
changes

```
@onlyNativeDeviceTypes
def test_empty_full(self, device):
    torch_device = torch.device(device)
    device_type = torch_device.type

    dtypes = get_all_dtypes(include_half=False, include_bfloat16=False, incl
    if device_type == 'cpu':
        do_test_empty_full(self, dtypes, torch.strided, torch_device)
    if device_type == 'cuda':
        do_test_empty_full(self, dtypes, torch.strided, None)
        do_test_empty_full(self, dtypes, torch.strided, torch_device)
```

```
# TODO: this test should be updated
@onlyNativeDeviceTypes
def test_empty_full(self, device):
    torch_device = torch.device(device)
    device_type = torch_device.type

    dtypes = get_all_dtypes(include_half=False, include_bfloat16=False, include_c
    if device_type == 'cpu':
        do_test_empty_full(self, dtypes, torch.strided, torch_device)
    if device_type != 'cpu':
        do_test_empty_full(self, dtypes, torch.strided, None)
        do_test_empty_full(self, dtypes, torch.strided, torch_device)

# TODO: this test should be updated
```

Replaced Hardcoded device
type strings

Test Refactoring: Results

```
if device_type == 'cuda':  
    self.assertEqual('cuda:0', str(torch.tensor(5).cuda(0).device))  
    self.assertEqual('cuda:0', str(torch.tensor(5).cuda('cuda:0').device))  
    self.assertEqual('cuda:0',  
                     str(torch.tensor(5, dtype=torch.int64, device=0).device))  
    self.assertEqual('cuda:0',  
                     str(torch.tensor(5, dtype=torch.int64, device='cuda:0').device))  
    self.assertEqual('cuda:0',  
                     str(torch.tensor(torch.ones((2, 3), dtype=torch.float32), device=0).device))  
    self.assertEqual('cuda:0', str(torch.tensor(np.random.randn(2, 3), dtype=torch.float32, device=0).device))  
  
    for device in devices:  
        with torch.cuda.device(device):  
            device_string = 'cuda:' + str(torch.cuda.current_device())  
            self.assertEqual(device_string,  
                             str(torch.tensor(5, dtype=torch.int64, device=device).device))  
  
        with self.assertRaises(RuntimeError):  
            torch.tensor(5).cuda('cpu')  
        with self.assertRaises(RuntimeError):  
            torch.tensor(5).cuda('cpu:0')  
  
if len(devices) > 1:  
    self.assertEqual('cuda:1', str(torch.tensor(5).cuda(1).device))  
    self.assertEqual('cuda:1', str(torch.tensor(5).cuda('cuda:1').device))  
    self.assertEqual('cuda:1',  
                     str(torch.tensor(5, dtype=torch.int64, device=1).device))  
    self.assertEqual('cuda:1',  
                     str(torch.tensor(5, dtype=torch.int64, device='cuda:1').device))  
    self.assertEqual('cuda:1',  
                     str(torch.tensor(torch.ones((2, 3), dtype=torch.float32), device='cuda:1').device))  
    self.assertEqual('cuda:1',  
                     str(torch.tensor(np.random.randn(2, 3), dtype=torch.float32, device='cuda:1').device))
```

```
# Check device handling for accelerators (CUDA, Spyre, etc.)  
if device_type != 'cpu':  
    # Use device-agnostic API calls  
    device_obj = torch.device(devices[0])  
    self.assertEqual(device_obj,  
                     torch.tensor(5, device=device_obj).device)  
    self.assertEqual(device_obj,  
                     torch.tensor(5, dtype=torch.int64, device=device_obj).device)  
    self.assertEqual(device_obj,  
                     torch.tensor(torch.ones((2, 3), dtype=torch.float32), device=device_obj).device)  
    self.assertEqual(device_obj,  
                     torch.tensor(np.random.randn(2, 3).astype(np.float32), device=device_obj).device)  
  
# Multi-device tests (if available)  
  
if len(devices) > 1:  
    device_obj2 = torch.device(devices[1])  
  
    self.assertEqual(device_obj2, torch.tensor(5, device=device_obj2).device)
```

Replaced hardcoded
device type
branches

Test Refactoring: Results

```
@onlyNativeDeviceTypes
@unittest.skipIf(IS_PPC, "Test is broken on PowerPC, see https://github.com/
@dtypes(torch.bool, torch.uint8, torch.int8, torch.int16, torch.int32, torch
def test_float_to_int_conversion_finite(self, device, dtype):
    min = torch.finfo(torch.float).min
    max = torch.finfo(torch.float).max

    # Note: CUDA max float -> integer conversion is divergent on some dtypes
    vals = (min, -2, -1.5, -.5, 0, .5, 1.5, 2, max)
    refs = None
    if self.device_type == 'cuda':
        if torch.version.hip:
            # HIP min float -> int64 conversion is divergent
            vals = (-2, -1.5, -.5, 0, .5, 1.5, 2)
        else:
            vals = (min, -2, -1.5, -.5, 0, .5, 1.5, 2)
    elif dtype == torch.uint8:
        # Note: CPU max float -> uint8 conversion is divergent
        vals = (min, -2, -1.5, -.5, 0, .5, 1.5, 2)
        # Note: numpy -2.0 or -1.5 -> uint8 conversion is undefined
        # see https://github.com/pytorch/pytorch/issues/97794
        refs = (0, 254, 255, 0, 0, 0, 1, 2)
    elif dtype == torch.int16:
        # CPU min and max float -> int16 conversion is divergent.
        vals = (-2, -1.5, -.5, 0, .5, 1.5, 2)
```

```
@onlyNativeDeviceTypes
@unittest.skipIf(IS_PPC, "Test is broken on PowerPC, see https://github.com/pyt
@dtypes(torch.bool, torch.uint8, torch.int8, torch.int16, torch.int32, torch.in
def test_float_to_int_conversion_finite(self, device, dtype):
    min = torch.finfo(torch.float).min
    max = torch.finfo(torch.float).max

    # Note: CUDA max float -> integer conversion is divergent on some dtypes
    vals = (min, -2, -1.5, -.5, 0, .5, 1.5, 2, max)
    refs = None
    if self.device_type == 'cuda': # Keep CUDA-specific for HIP compatibility
        if torch.version.hip:
            # HIP min float -> int64 conversion is divergent
            vals = (-2, -1.5, -.5, 0, .5, 1.5, 2)
        else:
            vals = (min, -2, -1.5, -.5, 0, .5, 1.5, 2)
    elif dtype == torch.uint8:
        # Note: CPU max float -> uint8 conversion is divergent
        vals = (min, -2, -1.5, -.5, 0, .5, 1.5, 2)
        # Note: numpy -2.0 or -1.5 -> uint8 conversion is undefined
        # see https://github.com/pytorch/pytorch/issues/97794
        refs = (0, 254, 255, 0, 0, 0, 1, 2)
    elif dtype == torch.int16:
        # CPU min and max float -> int16 conversion is divergent.
        vals = (-2, -1.5, -.5, 0, .5, 1.5, 2)
```

Our framework also identifies tests are specific to CUDA and does not refactor them

Test Refactoring: Results

```
@onlyCPU
@dtypes(torch.float)
def test_diagonal_multidim(self, device, dtype):
    x = torch.randn(10, 11, 12, 13, dtype=dtype, device=device)
    xn = x.numpy()
    for args in [(2, 2, 3), (2,), (-2, 1, 2), (0, -2, -1)]:
        result = torch.diagonal(x, *args)
        expected = xn.diagonal(*args)
        self.assertEqual(expected.shape, result.shape)
        self.assertEqual(expected, result)

    # test non-contiguous
    xp = x.permute(1, 2, 3, 0)
    result = torch.diagonal(xp, 0, -2, -1)
    expected = xp.numpy().diagonal(0, -2, -1)
    self.assertEqual(expected.shape, result.shape)
    self.assertEqual(expected, result)
```

```
@onlyNativeDeviceTypes
@dtypes(torch.float)
def test_diagonal_multidim(self, device, dtype):
    x = torch.randn(10, 11, 12, 13, dtype=dtype, device=device)
    xn = x.cpu().numpy()
    for args in [(2, 2, 3), (2,), (-2, 1, 2), (0, -2, -1)]:
        result = torch.diagonal(x, *args)
        expected = xn.diagonal(*args)
        self.assertEqual(expected.shape, result.shape)
        self.assertEqual(expected, result.cpu())

    # test non-contiguous
    xp = x.permute(1, 2, 3, 0)
    result = torch.diagonal(xp, 0, -2, -1)
    expected = xp.cpu().numpy().diagonal(0, -2, -1)
    self.assertEqual(expected.shape, result.shape)
    self.assertEqual(expected, result.cpu())
```

Our framework handles cases where the tests fail due to hardware limitations that are not integral to the semantics of the test by implementing CPU fallbacks

Test Refactoring: Results

```
class TestShapeOps(TestCase):
    # TODO: update to work on CUDA, too
    @onlyCPU
    def test_unbind(self, device):
        x = torch.rand(2, 3, 4, 5)
        for dim in range(4):
            res = torch.unbind(x, dim)
            res2 = x.unbind(dim)
            self.assertEqual(x.size(dim), len(res))
            self.assertEqual(x.size(dim), len(res2))
            for i in range(dim):
                self.assertEqual(x.select(dim, i), res[i])
                self.assertEqual(x.select(dim, i), res2[i])
```

```
class TestShapeOps(TestCase):
    # TODO: update to work on CUDA, too
    @onlyNativeDeviceTypes
    def test_unbind(self, device):
        x = torch.rand(2, 3, 4, 5, device=device)
        for dim in range(4):
            res = torch.unbind(x, dim)
            res2 = x.unbind(dim)
            self.assertEqual(x.size(dim), len(res))
            self.assertEqual(x.size(dim), len(res2))
            for i in range(dim):
                self.assertEqual(x.select(dim, i), res[i])
                self.assertEqual(x.select(dim, i), res2[i])
```

Updates tests such that it increases device coverage

Test Refactoring: Results

```
def is_view_of(self, base, other):  
    if (  
        not other._is_view()  
        or other is base  
        or other._base is not base  
        or base.device != other.device  
    ):  
        return False  
    # Note: only validates storage on native device types  
    # because some accelerators, like XLA, do not expose storage  
    if base.device.type in ["cpu", "cuda", "xpu"]:  
        if base.untyped_storage().data_ptr() != other.untyped_storage().data_ptr():  
            return False  
    return True
```

```
def is_view_of(self, base, other):  
    if (  
        not other._is_view()  
        or other is base  
        or other._base is not base  
        or base.device != other.device  
    ):  
        return False  
    # Note: only validates storage on native device types  
    # because some accelerators, like XLA, do not expose storage  
    if base.device.type in ["cpu", "cuda", "xpu", "spyre"]:  
        if base.untyped_storage().data_ptr() != other.untyped_storage().data_ptr():  
            return False  
    return True
```

Modifies test helper functions to properly accommodate the privateuse1 device

Test Refactoring: Results

```
test_suite_config:
  files:
    - path: ${TORCH_ROOT}/test/test_tensor_creation_ops.py
      unlisted_test_mode: skip
      tests:
        - names:
            - TestTensorCreation::test_tensor_device
            - TestTensorCreation::test_cat_stack_cross_devices
          mode: mandatory_success
          edits:
            functions:
              cpu_move:
                - name: assertEqual

    - path: ${TORCH_ROOT}/test/test_binary_ufuncs.py
      unlisted_test_mode: skip
      tests:
        - names:
            - TestBinaryUfuncs::test_division_by_scalar
          mode: mandatory_success

          edits:
            functions:
              cpu_move:
                - name: assertEqual

    - path: ${TORCH_ROOT}/test/test_shape_ops.py
      unlisted_test_mode: skip
      tests:
        - names:
            - TestShapeOps::test_diagonal_multidim
            - TestShapeOps::test_unbind
          mode: mandatory_success
          edits:
            functions:
              cpu_move:
                - name: assertEqual

  global:
    supported_dtypes:
      - name: float16
      - name: float32
      - name: bfloat16
```

Before

```
Running 2 items in this shard
test_tensor_creation_ops__oot_wrapper.py::TestTensorCreationPRIVATEUSE1::test_cat_stack_cross_devices_spyre PASSED [0.0769s] [ 50%]
test_tensor_creation_ops__oot_wrapper.py::TestTensorCreationPRIVATEUSE1::test_tensor_device_spyre PASSED [0.0009s] [100%]
```

`test_cat_stack_cross_devices` is handled by test adaptation framework but it works for privateuse1 but not for other accelerators other than CUDA.

`test_tensor_device` is passed but it doesn't run on spyre.

```
Running 2 items in this shard
test_shape_ops.py::TestShapeOpsPRIVATEUSE1::test_diagonal_multidim_spyre_float32 FAILED [0.0075s] [ 50%]
test_shape_ops.py::TestShapeOpsPRIVATEUSE1::test_unbind_spyre PASSED [0.0029s] [100%]
```

`test_diagonal_multidim` fails because numpy arrays can't be directly converted to torch tensor in spyre, they need to be moved to cpu first.

`test_unbind` is passed but it doesn't run on spyre.

```
Running 2 items in this shard
test_binary_ufuncs.py::TestBinaryUfuncsPRIVATEUSE1::test_division_by_scalar_spyre_bfloat16 PASSED [5.4667s] [ 50%]
test_binary_ufuncs.py::TestBinaryUfuncsPRIVATEUSE1::test_division_by_scalar_spyre_float32 PASSED [6.3245s] [100%]
```

`test_division_by_scalar` is handled by test adaptation framework but it works for privateuse1 but not for other accelerators other than CUDA. It also only test bfloat16 and float32 but the test can also support float16.

We use this config to verify the refactored results

Test Refactoring: Results

TestTensorCreation::test_stack_cross_device

```
- @onlyCUDA
+ @onlyNativeDeviceTypes
def test_cat_stack_cross_devices(self, device):
    cuda = torch.randn((3, 3), device=device)
    cpu = torch.randn((3, 3), device='cpu')

    # Stack
    with self.assertRaisesRegex(RuntimeError,
                                "Expected all tensors to be on the same device"):
        torch.stack((cuda, cpu))
    with self.assertRaisesRegex(RuntimeError,
                                "Expected all tensors to be on the same device"):
        torch.stack((cpu, cuda))
```

Modified decorator to run on all native devices

TestBinaryUfuncs::test_divison_by_scalar

```
- @onlyCUDA
- @dtypes(torch.float, torch.bfloat16)
+ @onlyNativeDeviceTypes
+ @dtypes(torch.float, torch.float16, torch.bfloat16)
def test_division_by_scalar(self, device, dtype):
    num = torch.rand(1024, device=device, dtype=dtype)
    denom = torch.logspace(-4, 4, steps=20)
    denom = [d.item() for d in denom]
    res = [num / d for d in denom]
    ref = [num * (1 / d) for d in denom]

    # res = [(num / d).cpu() for d in denom]
    # ref = [(num * (1 / d)).cpu() for d in denom]

    self.assertEqual(res, ref, atol=0, rtol=0)
```

Refactored to extend dtype coverage to include float16

Test Refactoring: Results

TestTensorCreation::test_tensor_device

```
@deviceCountAtLeast(1)
def test_tensor_device(self, devices):
    device_type = torch.device(devices[0]).type
    # Check device handling for CPU
    if device_type == 'cpu':
        self.assertEqual('cpu', torch.tensor(5).device.type)
        self.assertEqual('cpu',
            torch.ones((2, 3), dtype=torch.float32, device='cpu').device.type)
        self.assertEqual('cpu',
            torch.ones((2, 3), dtype=torch.float32, device='cpu:0').device.type)
        self.assertEqual('cpu',
            torch.tensor(torch.ones((2, 3), dtype=torch.float32), device='cpu:0').device.type)
        self.assertEqual('cpu', torch.tensor(np.random.randn(2, 3), device='cpu').device.type)
    if device_type == 'cuda':
        self.assertEqual('cuda:0', str(torch.tensor(5).cuda(0).device))
        self.assertEqual('cuda:0', str(torch.tensor(5).cuda('cuda:0').device))
        self.assertEqual('cuda:0',
            str(torch.tensor(5, dtype=torch.int64, device=0).device))
        self.assertEqual('cuda:0',
            str(torch.tensor(5, dtype=torch.int64, device='cuda:0').device))
        self.assertEqual('cuda:0',
            str(torch.tensor(torch.ones((2, 3), dtype=torch.float32), device='cuda:0').device))
        self.assertEqual('cuda:0', str(torch.tensor(np.random.randn(2, 3), device='cuda:0').device))
    for device in devices:
        with torch.cuda.device(device):
            device_string = 'cuda:' + str(torch.cuda.current_device())
            self.assertEqual(device_string,
                str(torch.tensor(5, dtype=torch.int64, device='cuda').device))
    with self.assertRaises(RuntimeError):
        torch.tensor(5).cuda('cpu')
    with self.assertRaises(RuntimeError):
        torch.tensor(5).cuda('cpu:0')
```

```
+ if device_type != 'cpu':
+     # Use device-agnostic API calls
+     device_obj = torch.device(devices[0])
+     self.assertEqual(device_obj,
+         torch.tensor(5, device=device_obj).device)
+     self.assertEqual(device_obj,
+         torch.tensor(5, dtype=torch.int64, device=device_obj).device)
+     self.assertEqual(device_obj,
+         torch.tensor(torch.ones((2, 3), dtype=torch.float32), device=device_obj).device)
+     self.assertEqual(device_obj,
+         torch.tensor(np.random.randn(2, 3).astype(np.float32), device=device_obj).device)
+
+     # Multi-device tests (if available)
+     if len(devices) > 1:
-         self.assertEqual('cuda:1', str(torch.tensor(5).cuda(1).device))
-         self.assertEqual('cuda:1', str(torch.tensor(5).cuda('cuda:1').device))
-         self.assertEqual('cuda:1',
-             str(torch.tensor(5, dtype=torch.int64, device=1).device))
-         self.assertEqual('cuda:1',
-             str(torch.tensor(5, dtype=torch.int64, device='cuda:1').device))
-         self.assertEqual('cuda:1',
-             str(torch.tensor(torch.ones((2, 3), dtype=torch.float32),
-                 device='cuda:1').device))
-         self.assertEqual('cuda:1',
-             str(torch.tensor(np.random.randn(2, 3), device='cuda:1').device))
+         device_obj2 = torch.device(devices[1])
+         self.assertEqual(device_obj2, torch.tensor(5, device=device_obj2).device)
```

Refactored to use the device provided by the test arguments, simplifying the test and enabling Spyre support.

Test Refactoring: Results

TestShapeOps::test_diagonal_multidim

```
- @onlyCPU
+ @onlyNativeDeviceTypes
  @dtypes(torch.float)
  def test_diagonal_multidim(self, device, dtype):
    x = torch.randn(10, 11, 12, 13, dtype=dtype, device=device)
-    xn = x.numpy()
+    xn = x.cpu().numpy()
    for args in [(2, 2, 3), (2,), (-2, 1, 2), (0, -2, -1)]:
      result = torch.diagonal(x, *args)
      expected = xn.diagonal(*args)
      self.assertEqual(expected.shape, result.shape)
-      self.assertEqual(expected, result)
+      self.assertEqual(expected, result.cpu())

    # test non-contiguous
    xp = x.permute(1, 2, 3, 0)
    result = torch.diagonal(xp, 0, -2, -1)
-    expected = xp.numpy().diagonal(0, -2, -1)
+    expected = xp.cpu().numpy().diagonal(0, -2, -1)
    self.assertEqual(expected.shape, result.shape)
-    self.assertEqual(expected, result)
+    self.assertEqual(expected, result.cpu())
```

Refactored to work around Spyre's NumPy-to-tensor limitation by staging through CPU.

TestShapeOps::test_unbind

```
- @onlyCPU
+ @onlyNativeDeviceTypes
  def test_unbind(self, device):
-    x = torch.rand(2, 3, 4, 5)
+    x = torch.rand(2, 3, 4, 5, device=device)
    for dim in range(4):
      res = torch.unbind(x, dim)
      res2 = x.unbind(dim)
```

Refactored to use the device provided by the test arguments, and enabling Spyre support.

Test Refactoring: Results

```
test_suite_config:
  files:
    - path: ${TORCH_ROOT}/test/test_tensor_creation_ops.py
      unlisted_test_mode: skip
      tests:
        - names:
            - TestTensorCreation::test_tensor_device
            - TestTensorCreation::test_cat_stack_cross_devices
          mode: mandatory_success
          edits:
            functions:
              cpu_move:
                - name: assertEqual

    - path: ${TORCH_ROOT}/test/test_binary_ufuncs.py
      unlisted_test_mode: skip
      tests:
        - names:
            - TestBinaryUfuncs::test_division_by_scalar
          mode: mandatory_success

          edits:
            functions:
              cpu_move:
                - name: assertEqual

    - path: ${TORCH_ROOT}/test/test_shape_ops.py
      unlisted_test_mode: skip
      tests:
        - names:
            - TestShapeOps::test_diagonal_multidim
            - TestShapeOps::test_unbind
          mode: mandatory_success
          edits:
            functions:
              cpu_move:
                - name: assertEqual

  global:
    supported_dtypes:
      - name: float16
      - name: float32
      - name: bfloat16
```

We use this config to verify the refactored results

After

```
collected 2 items
Running 2 items in this shard

test_tensor_creation_ops__oot_wrapper.py::TestTensorCreationPRIVATEUSE1::test_cat_stack_cross_devices_spyre PASSED [0.0812s] [ 50%]
test_tensor_creation_ops__oot_wrapper.py::TestTensorCreationPRIVATEUSE1::test_tensor_device_spyre PASSED [0.0022s] [100%]
```

Refactored to use backend-agnostic device handling and decorators, enabling successful execution on the Spyre accelerator.

```
collected 2 items
Running 2 items in this shard

test_shape_ops.py::TestShapeOpsPRIVATEUSE1::test_diagonal_multidim_spyre_float32 PASSED [0.0125s] [ 50%]
test_shape_ops.py::TestShapeOpsPRIVATEUSE1::test_unbind_spyre PASSED [0.0051s] [100%]
```

Refactored to handle Spyre-specific limitations, ensuring compatibility with the Spyre accelerator.

```
collected 3 items
Running 3 items in this shard

test_binary_ufuncs.py::TestBinaryUfuncsPRIVATEUSE1::test_division_by_scalar_spyre_bfloat16 PASSED [6.1273s] [ 33%]
test_binary_ufuncs.py::TestBinaryUfuncsPRIVATEUSE1::test_division_by_scalar_spyre_float16 PASSED [5.8137s] [ 66%]
test_binary_ufuncs.py::TestBinaryUfuncsPRIVATEUSE1::test_division_by_scalar_spyre_float32 PASSED [7.2418s] [100%]
```

Refactored to expand supported dtype coverage and adopt a device-generic decorator.